



university of  
 groningen

# Models and Semantics of Computation (MaSC)

## Lecture 9: Concurrency and CCS

Jorge A. Pérez

Bernoulli Institute for Mathematics, Computer Science, and AI  
University of Groningen, Groningen, the Netherlands

# Today: Concurrency



Sequential computation (IMP/HOFL)

- ▶ determinacy
- ▶ any two non-terminating programs are equivalent

# Today: Concurrency



## Sequential computation (IMP/HOFL)

- ▶ determinacy
- ▶ any two non-terminating programs are equivalent

## Concurrent computation (CCS and $\pi$ )

- ▶ exhibit intrinsic nondeterminism to external observers
- ▶ non-termination can be a desirable feature (e.g. servers)
- ▶ not all non-terminating processes are equivalent
- ▶ interaction is a primary issue
- ▶ new notions of behavior / equivalence are needed

# CCS: Calculus of Communicating Systems



## Process Calculi (aka Process Algebras)

- ▶ focus on few primitive operators (essential features)
- ▶ concise syntax to construct and compose **processes**
- ▶ not a full-fledged programming language
- ▶ full computational power (Turing equivalent)

# CCS: Calculus of Communicating Systems



## Process Calculi (aka Process Algebras)

- ▶ focus on few primitive operators (essential features)
- ▶ concise syntax to construct and compose **processes**
- ▶ not a full-fledged programming language
- ▶ full computational power (Turing equivalent)

## Communication

- ▶ binary, message-passing over channels

# CCS: Calculus of Communicating Systems



## Process Calculi (aka Process Algebras)

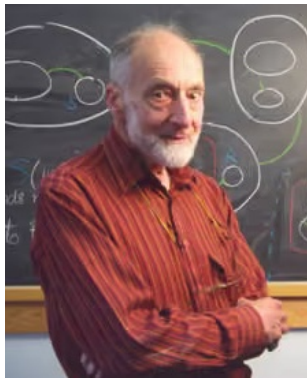
- ▶ focus on few primitive operators (essential features)
- ▶ concise syntax to construct and compose **processes**
- ▶ not a full-fledged programming language
- ▶ full computational power (Turing equivalent)

## Communication

- ▶ binary, message-passing over channels

## Structural Operational Semantics

- ▶ small-step style (Labeled Transition System)
- ▶ processes as states, ongoing interactions as labels
- ▶ defined by inference rules



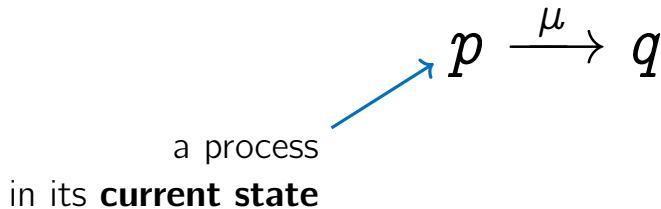
**Robin Milner**  
(Turing award 1991)

# Labeled Transitions



$$p \xrightarrow{\mu} q$$

# Labeled Transitions



# Labeled Transitions



**ongoing interaction** with  
the environment (other processes)



$$p \xrightarrow{\mu} q$$

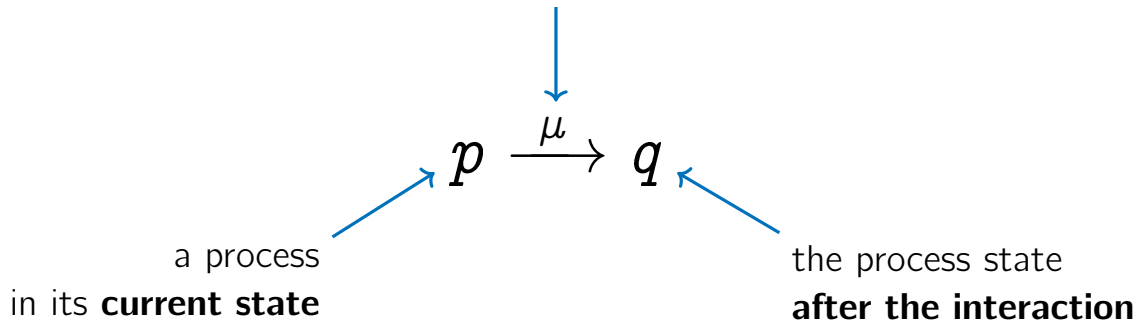


a process  
in its **current state**

# Labeled Transitions



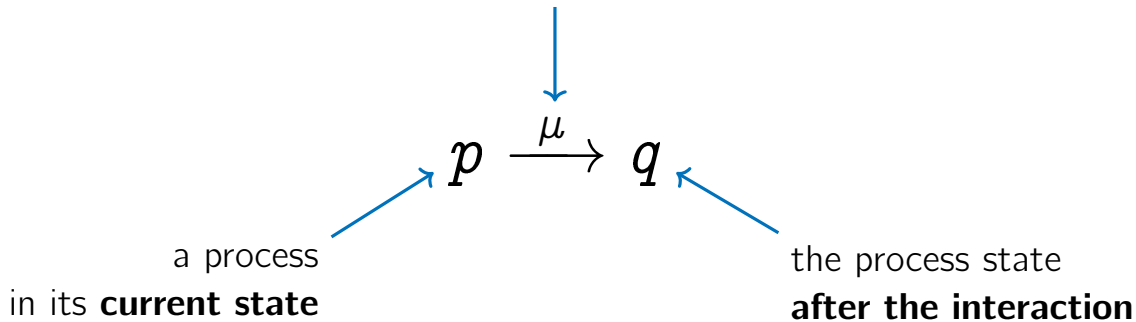
**ongoing interaction** with  
the environment (other processes)



# Labeled Transitions

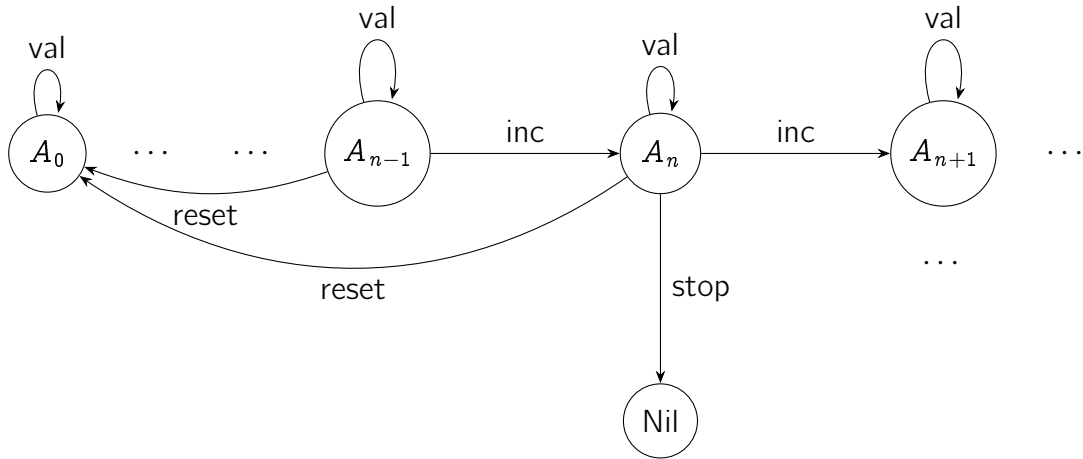


**ongoing interaction** with  
the environment (other processes)

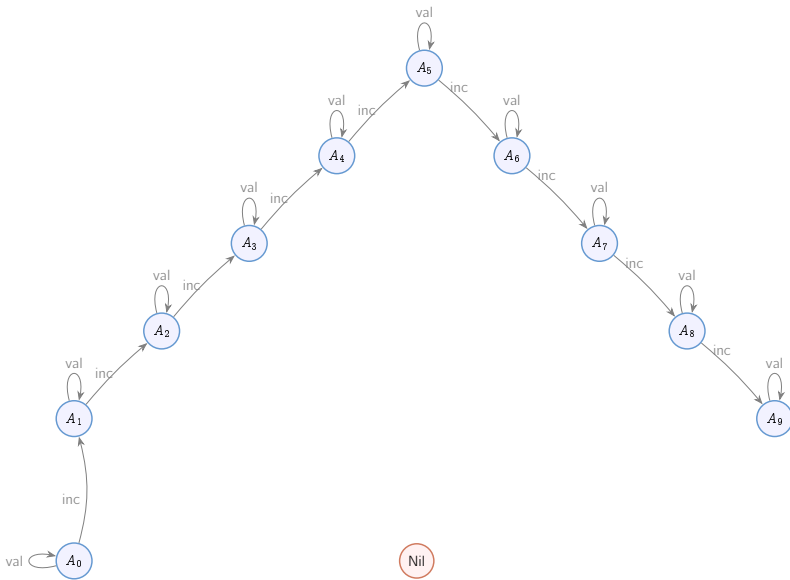


The number of states/transitions can be **infinite!**

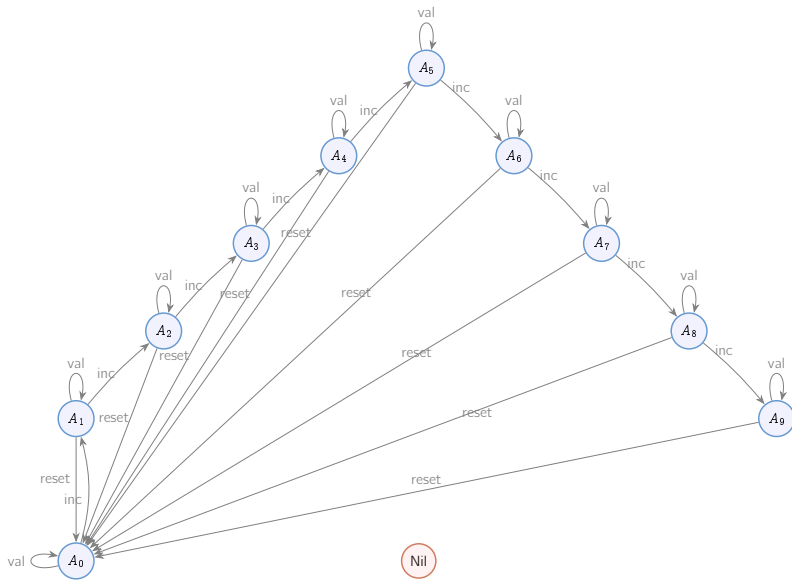
# Example: Counter



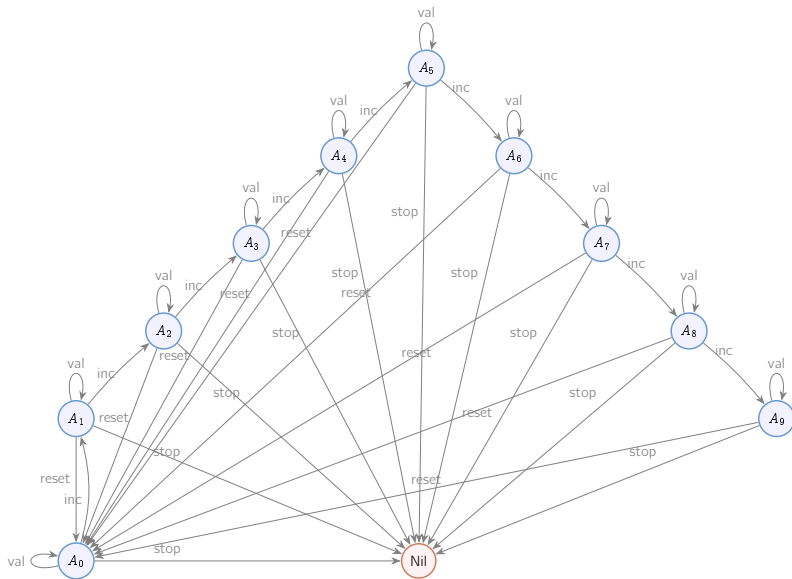
# Labeled Transition System



# Labeled Transition System



# Labeled Transition System



# CCS: States and Labels



$$p \xrightarrow{\mu} q$$

What is a **process**  $p$ ?

- ▶ a sequential agent
- ▶ a system where many sequential agents interact

# CCS: States and Labels



$$p \xrightarrow{\mu} q$$

What is a **process**  $p$ ?

- ▶ a sequential agent
- ▶ a system where many sequential agents interact

What is a **label**  $\mu$ ?

- ▶ an **action** (e.g. an output)

$\alpha!v$  (**send** value  $v$  on channel  $\alpha$ )



$$p \xrightarrow{\mu} q$$

What is a **process**  $p$ ?

- ▶ a sequential agent
- ▶ a system where many sequential agents interact

What is a **label**  $\mu$ ?

- ▶ an **action** (e.g. an output)
- ▶ a **dual** action (e.g. an input)

$\alpha!v$  (**send** value  $v$  on channel  $\alpha$ )

$\alpha?v$  (**receive** value  $v$  on channel  $\alpha$ )



$$p \xrightarrow{\mu} q$$

What is a **process**  $p$ ?

- ▶ a sequential agent
- ▶ a system where many sequential agents interact

What is a **label**  $\mu$ ?

- ▶ an **action** (e.g. an output)
- ▶ a **dual** action (e.g. an input)
- ▶ an **internal** action (silent action)  
no interaction with the environment

$\alpha!v$  (**send** value  $v$  on channel  $\alpha$ )

$\alpha?v$  (**receive** value  $v$  on channel  $\alpha$ )

$\tau$  (concluded communication)



- We can be even more abstract without losing computational expressiveness
- We may disregard communicated values (assuming a dedicated channel for each value). This way:

$\alpha!v$  becomes  $\overline{\alpha}_v$  or just  $\overline{\alpha}$

$\alpha?v$  becomes  $\alpha_v$  or just  $\alpha$

$\lambda$  denotes either  $\overline{\alpha}$  or  $\alpha$

$\overline{\lambda}$  denotes its dual (assume  $\overline{\overline{\alpha}} = \alpha$ )

# Communication in CCS



Given processes  $p_1$  and  $p_2$  such that

$$p_1 \xrightarrow{\lambda} q_1$$

$$p_2 \xrightarrow{\bar{\lambda}} q_2$$

We may **compose** them in parallel

$$p_1 \mid p_2$$

# Communication in CCS



Given processes  $p_1$  and  $p_2$  such that

$$p_1 \xrightarrow{\lambda} q_1$$

$$p_2 \xrightarrow{\bar{\lambda}} q_2$$

We may **compose** them in parallel to observe their **interaction**:

$$p_1 \mid p_2 \xrightarrow{\tau}$$

# Communication in CCS



Given processes  $p_1$  and  $p_2$  such that

$$p_1 \xrightarrow{\lambda} q_1$$

$$p_2 \xrightarrow{\bar{\lambda}} q_2$$

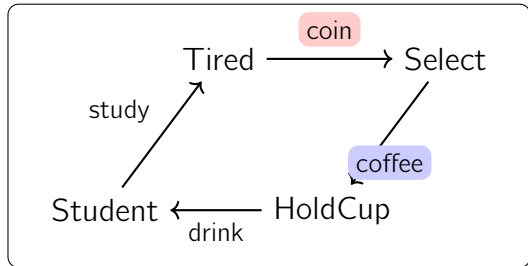
We may **compose** them in parallel to observe their **interaction**:

$$p_1 \mid p_2 \xrightarrow{\tau} q_1 \mid q_2$$

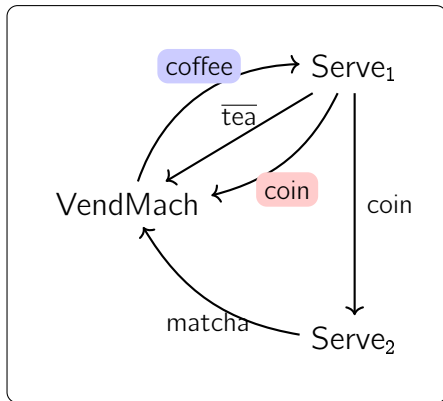
# Example: Student and Vending Machine



## Student Automaton



## Vending Machine Automaton





|                         |                                  |
|-------------------------|----------------------------------|
| $p, q ::= \text{nil}$   | inactive process                 |
| $  x$                   | process variable (for recursion) |
| $  \mu.p$               | action prefix                    |
| $  p \backslash \alpha$ | restricted channel               |
| $  p[\phi]$             | channel relabeling               |
| $  p + q$               | non deterministic choice         |
| $  p \mid q$            | parallel composition             |
| $  \text{rec } x. p$    | recursion                        |

(Note: operators are listed in order of precedence)



$$p, q ::= \text{nil} \mid x \mid \mu.p \mid p \backslash \alpha \mid p[\phi] \mid p + q \mid p \mid q \mid \text{rec } x.p$$

## Example (Precedence)

The process

$$p \triangleq \text{rec } x. \text{coffee}.x + \text{tea}. \text{nil} \mid \text{water}. \text{nil}$$

is to be read as

$$\text{rec } x. (((\text{coffee}.x) + \text{tea}. \text{nil}) \mid \text{water}. \text{nil})$$

What does  $p$  specify?



- ▶ The only binder is the **recursion operator**:  $\text{rec } x. p$ .
- ▶ Given  $p$ , the set of (free) process variables is defined as expected, and denoted  $\text{fv}(p)$ . A process is **closed** if it has no free variables.



- ▶ The only binder is the **recursion operator**:  $\text{rec } x. p$ .
- ▶ Given  $p$ , the set of (free) process variables is defined as expected, and denoted  $\text{fv}(p)$ . A process is **closed** if it has no free variables.
- ▶ **Capture-avoiding substitution** is defined as usual, and denoted  $p[q/x]$ .
- ▶ Processes are taken up-to alpha-renaming of bound variables.

Example:

$$\text{rec } x. \text{coffee}.x = \text{rec } y. \text{coffee}.y$$



# Operational Semantics



$\mathcal{C}$  set of (input) actions, ranged over by  $\alpha$   
 $\overline{\mathcal{C}}$  set of (output) actions, ranged over by  $\overline{\alpha}$



$\mathcal{C}$  set of (input) actions, ranged over by  $\alpha$   
 $\bar{\mathcal{C}}$  set of (output) actions, ranged over by  $\bar{\alpha}$   
 $\mathcal{C} \cap \bar{\mathcal{C}} = \emptyset$



$\mathcal{C}$  set of (input) actions, ranged over by  $\alpha$

$\bar{\mathcal{C}}$  set of (output) actions, ranged over by  $\bar{\alpha}$

$$\mathcal{C} \cap \bar{\mathcal{C}} = \emptyset$$

$\Lambda = \mathcal{C} \cup \bar{\mathcal{C}}$  set of **observable actions**, ranged over by  $\lambda, \bar{\lambda}$



$\mathcal{C}$  set of (input) actions, ranged over by  $\alpha$

$\bar{\mathcal{C}}$  set of (output) actions, ranged over by  $\bar{\alpha}$

$$\mathcal{C} \cap \bar{\mathcal{C}} = \emptyset$$

$\Lambda = \mathcal{C} \cup \bar{\mathcal{C}}$  set of **observable actions**, ranged over by  $\lambda, \bar{\lambda}$

$\tau \notin \Lambda$  a distinguished **silent action**



$\mathcal{C}$  set of (input) actions, ranged over by  $\alpha$

$\bar{\mathcal{C}}$  set of (output) actions, ranged over by  $\bar{\alpha}$

$$\mathcal{C} \cap \bar{\mathcal{C}} = \emptyset$$

$\Lambda = \mathcal{C} \cup \bar{\mathcal{C}}$  set of **observable actions**, ranged over by  $\lambda, \bar{\lambda}$

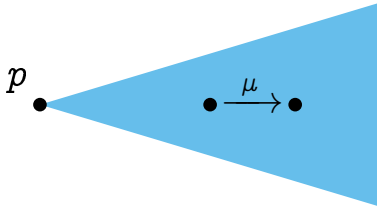
$\tau \notin \Lambda$  a distinguished **silent action**

$\mathcal{L} = \Lambda \cup \tau$  set of **actions**, ranged over  $\mu$

# LTS of a Process



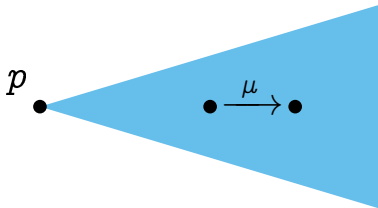
The LTS of CCS is **infinite**: there is one state per process. Graphically:



# LTS of a Process



The LTS of CCS is **infinite**: there is one state per process. Graphically:



Starting from  $p$ , consider all reachable states: the LTS of a process can be finite/infinite.

We introduce the **inference rules** for  $p \xrightarrow{\mu} q$ , considering each process construct.

# The Nil Process



$\text{nil} \not\rightarrow$

- ▶ the inactive process represents a terminated agent that does nothing
- ▶ no interaction is possible with the environment
- ▶ no associated semantic rule

# LTS of a Process



nil

# Action Prefix



act

$$\frac{}{\mu.p \xrightarrow{\mu} p}$$

- ▶ An action-prefixed process can perform an action  $\mu \in \mathcal{L} \cup \tau$  and then continue as specified as  $p$ .
- ▶ The action may involve an interaction with the environment.

# Action Prefix



act

$$\frac{}{\mu.p \xrightarrow{\mu} p}$$

- ▶ An action-prefixed process can perform an action  $\mu \in \mathcal{L} \cup \tau$  and then continue as specified as  $p$ .
- ▶ The action may involve an interaction with the environment.

## Example

The process

$coin.\overline{coffee}.nil$

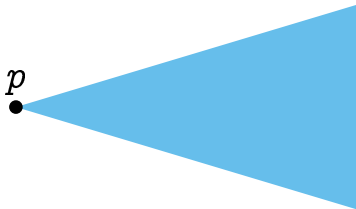
waits a coin, then gives a coffee, and then it stops:

$$coin.\overline{coffee}.nil \xrightarrow{coin} \overline{coffee}.nil \xrightarrow{\overline{coffee}} nil$$

# LTS of a Process



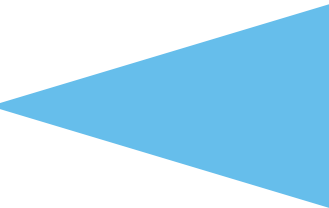
$p$



# LTS of a Process



$$\mu.p \bullet \xrightarrow{\mu} p \bullet$$



# Non-Deterministic Choice



SumL

$$\frac{p_1 \xrightarrow{\mu} q}{p_1 + p_2 \xrightarrow{\mu} q}$$

SumR

$$\frac{p_2 \xrightarrow{\mu} q}{p_1 + p_2 \xrightarrow{\mu} q}$$

- Process  $p_1 + p_2$  can behave either as  $p_1$  or as  $p_2$ .

# Non-Deterministic Choice



SumL

$$\frac{p_1 \xrightarrow{\mu} q}{p_1 + p_2 \xrightarrow{\mu} q}$$

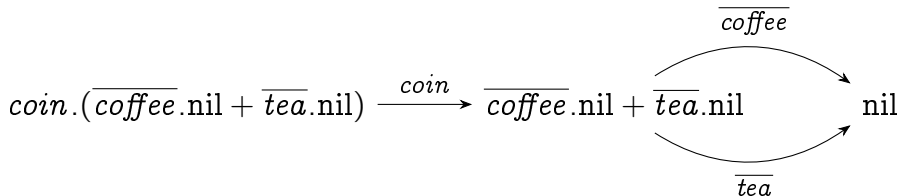
SumR

$$\frac{p_2 \xrightarrow{\mu} q}{p_1 + p_2 \xrightarrow{\mu} q}$$

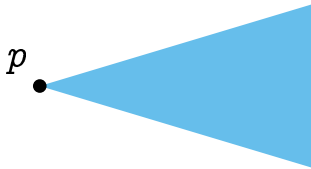
- Process  $p_1 + p_2$  can behave either as  $p_1$  or as  $p_2$ .

## Example

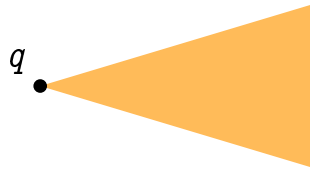
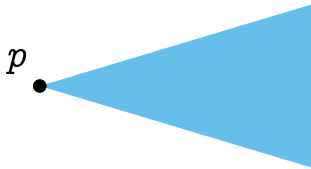
The process  $\text{coin} . (\overline{\text{coffee}}.\text{nil} + \overline{\text{tea}}.\text{nil})$  waits a coin, then gives a coffee or a tea, and then it stops:



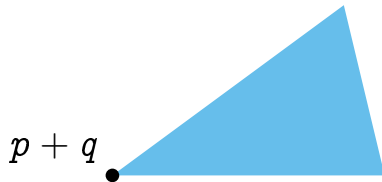
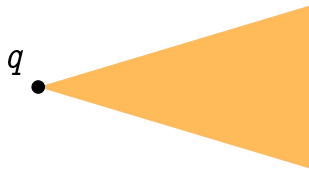
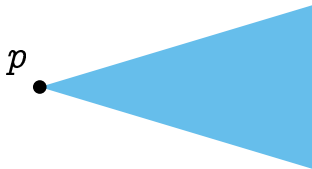
# LTS of a Process



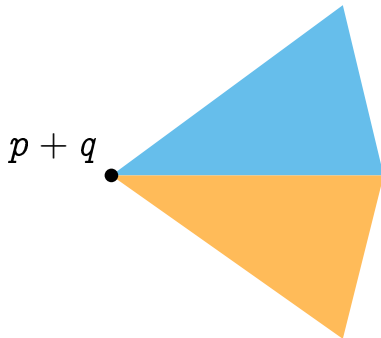
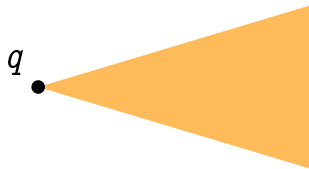
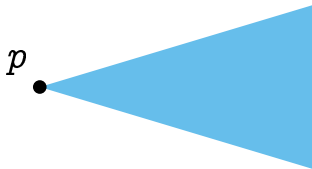
# LTS of a Process



# LTS of a Process



# LTS of a Process



# Recursion



$$\text{Rec} \frac{p[\text{rec } x. p/x] \xrightarrow{\mu} q}{\text{rec } x. p \xrightarrow{\mu} q}$$

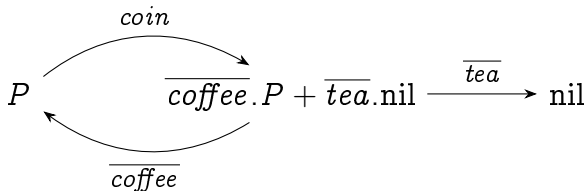
- Similar to a recursive definition 'let  $x = p$  in  $x$ '.

$$\text{Rec} \frac{p[\text{rec } x. p/x] \xrightarrow{\mu} q}{\text{rec } x. p \xrightarrow{\mu} q}$$

- Similar to a recursive definition 'let  $x = p$  in  $x$ '.

## Example

The process  $P \triangleq \text{rec } x. \text{coin}.(\overline{\text{coffee}}.x + \overline{\text{tea}}.\text{nil})$  waits a coin, then gives a coffee and is ready again **or** gives a tea and stops:



# Recursion via Process Constants



- Suppose some process constants  $A$  are defined together with a set of declarations  $\Delta$  of the form

$$A \triangleq p$$

one per constant.

- We would have the rule

$$\frac{\text{Const} \quad (A \triangleq p) \in \Delta \quad p \xrightarrow{\mu} q}{A \xrightarrow{\mu} q}$$

# Recursion via Process Constants



- Suppose some process constants  $A$  are defined together with a set of declarations  $\Delta$  of the form

$$A \triangleq p$$

one per constant.

- We would have the rule

$$\frac{\text{Const} \quad (A \triangleq p) \in \Delta \quad p \xrightarrow{\mu} q}{A \xrightarrow{\mu} q}$$

- The previous example:

$$P \triangleq \text{coin} . (\overline{\text{coffee}} . P + \overline{\text{tea}} . \text{nil})$$

# Example: A Buffer of Capacity 1



- Suppose

$$\Delta = \{B_0^1 \triangleq in.B_1^1, B_1^1 \triangleq \overline{out}.B_0^1\}$$

Intuition: in state  $B_j^i$ ,  $i$  denotes the capacity and  $j$  the usage.  
The buffer has availability if  $j < i$ .

- This is analogous to defining `rec  $x$ . in.  $\overline{out}.x$`

# Example: A Buffer of Capacity 1

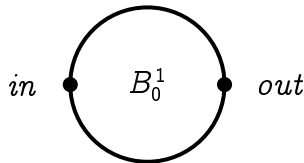
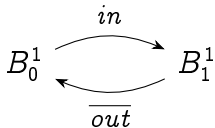


- Suppose

$$\Delta = \{B_0^1 \triangleq in.B_1^1, B_1^1 \triangleq \overline{out}.B_0^1\}$$

Intuition: in state  $B_j^i$ ,  $i$  denotes the capacity and  $j$  the usage.  
The buffer has availability if  $j < i$ .

- This is analogous to defining `rec  $x$ .  $in.\overline{out}.x$`
- Graphically:



## Example: A Buffer of Capacity 2



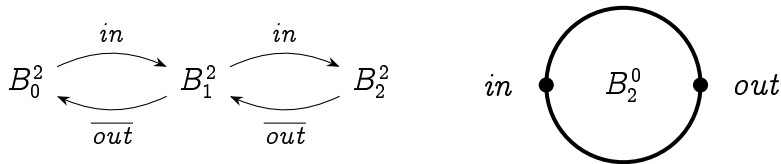
- The extension to capacity 2 uses an additional state (“half-empty”):

$$B_0^2 \triangleq in.B_1^2$$

$$B_1^2 \triangleq in.B_2^2 + \overline{out}.B_0^2$$

$$B_2^2 \triangleq \overline{out}.B_1^2$$

- Graphically:



# Example: A Boolean Buffer



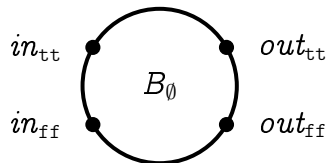
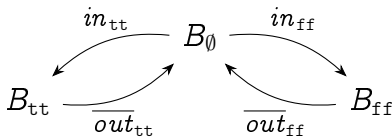
- We have states for storing true ( $tt$ ) and false ( $ff$ ):

$$B_{\emptyset} \triangleq in_{tt}.B_{tt} + in_{ff}.B_{ff}$$

$$B_{tt} \triangleq \overline{out_{tt}}.B_{\emptyset}$$

$$B_{ff} \triangleq \overline{out_{ff}}.B_{\emptyset}$$

- Graphically:



# Parallel Composition



ParL

$$\frac{p_1 \xrightarrow{\mu} q_1}{p_1 \mid p_2 \xrightarrow{\mu} q_1 \mid p_2}$$

Com

$$\frac{p_1 \xrightarrow{\lambda} q_1 \quad p_2 \xrightarrow{\bar{\lambda}} q_2}{p_1 \mid p_2 \xrightarrow{\tau} q_1 \mid q_2}$$

ParR

$$\frac{p_2 \xrightarrow{\mu} q_2}{p_1 \mid p_2 \xrightarrow{\mu} p_1 \mid p_2}$$

- Processes running in parallel can interleave their actions or synchronize when dual actions are performed.

## Example

Given  $P \triangleq \overline{coin}.coffee.nil$  and  $M \triangleq coin.(\overline{coffee}.nil + \overline{tea}.nil)$ , we have:

# Parallel Composition



ParL

$$\frac{p_1 \xrightarrow{\mu} q_1}{p_1 \mid p_2 \xrightarrow{\mu} q_1 \mid p_2}$$

Com

$$\frac{p_1 \xrightarrow{\lambda} q_1 \quad p_2 \xrightarrow{\bar{\lambda}} q_2}{p_1 \mid p_2 \xrightarrow{\tau} q_1 \mid q_2}$$

ParR

$$\frac{p_2 \xrightarrow{\mu} q_2}{p_1 \mid p_2 \xrightarrow{\mu} p_1 \mid p_2}$$

- Processes running in parallel can interleave their actions or synchronize when dual actions are performed.

## Example

Given  $P \triangleq \overline{coin}.coffee.nil$  and  $M \triangleq coin.(\overline{coffee}.nil + \overline{tea}.nil)$ , we have:

- $P \mid M \xrightarrow{\overline{coin}} coffee.nil \mid M$  ( $P$  moves independently)

# Parallel Composition



ParL

$$\frac{p_1 \xrightarrow{\mu} q_1}{p_1 \mid p_2 \xrightarrow{\mu} q_1 \mid p_2}$$

Com

$$\frac{p_1 \xrightarrow{\lambda} q_1 \quad p_2 \xrightarrow{\bar{\lambda}} q_2}{p_1 \mid p_2 \xrightarrow{\tau} q_1 \mid q_2}$$

ParR

$$\frac{p_2 \xrightarrow{\mu} q_2}{p_1 \mid p_2 \xrightarrow{\mu} p_1 \mid p_2}$$

- Processes running in parallel can interleave their actions or synchronize when dual actions are performed.

## Example

Given  $P \triangleq \overline{coin}.coffee.nil$  and  $M \triangleq coin.(\overline{coffee}.nil + \overline{tea}.nil)$ , we have:

- $P \mid M \xrightarrow{\overline{coin}} coffee.nil \mid M$  ( $P$  moves independently)
- $P \mid M \xrightarrow{coin} P \mid \overline{coffee}.nil + \overline{tea}.nil$  ( $M$  moves independently)

# Parallel Composition



ParL

$$\frac{p_1 \xrightarrow{\mu} q_1}{p_1 \mid p_2 \xrightarrow{\mu} q_1 \mid p_2}$$

Com

$$\frac{p_1 \xrightarrow{\lambda} q_1 \quad p_2 \xrightarrow{\bar{\lambda}} q_2}{p_1 \mid p_2 \xrightarrow{\tau} q_1 \mid q_2}$$

ParR

$$\frac{p_2 \xrightarrow{\mu} q_2}{p_1 \mid p_2 \xrightarrow{\mu} p_1 \mid p_2}$$

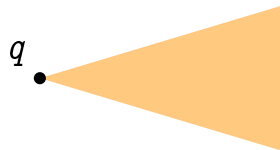
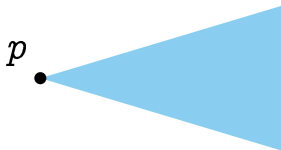
- Processes running in parallel can interleave their actions or synchronize when dual actions are performed.

## Example

Given  $P \triangleq \overline{coin}.coffee.nil$  and  $M \triangleq coin.(\overline{coffee}.nil + \overline{tea}.nil)$ , we have:

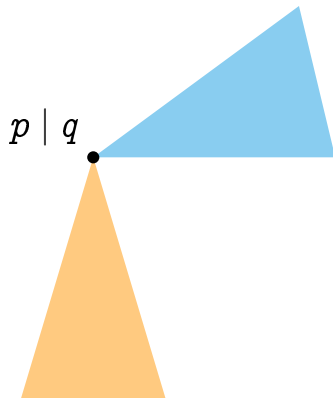
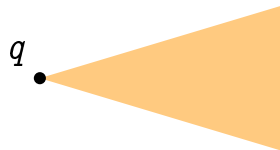
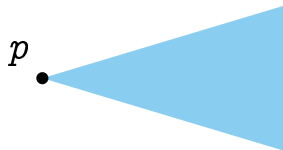
- $P \mid M \xrightarrow{\overline{coin}} coffee.nil \mid M$  ( $P$  moves independently)
- $P \mid M \xrightarrow{coin} P \mid \overline{coffee}.nil + \overline{tea}.nil$  ( $M$  moves independently)
- $P \mid M \xrightarrow{\tau} coffee.nil \mid \overline{coffee}.nil + \overline{tea}.nil$  ( $P$  and  $M$  synchronize)

# LTS of a Process

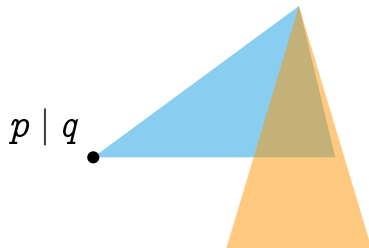
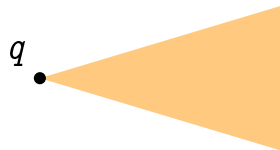
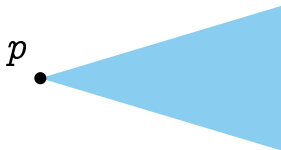


$p \mid q$  •

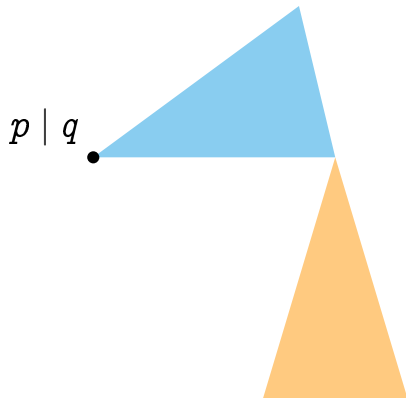
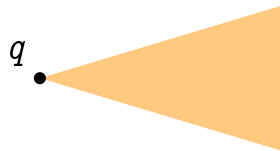
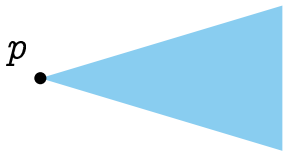
# LTS of a Process



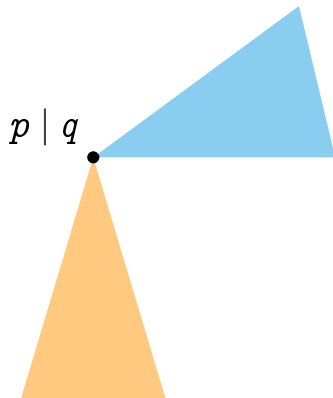
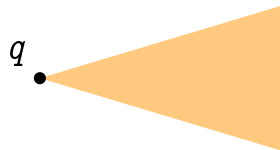
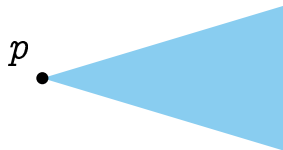
# LTS of a Process



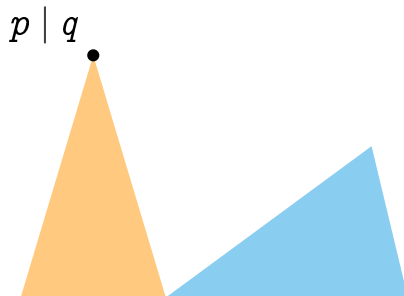
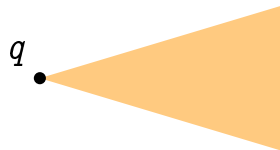
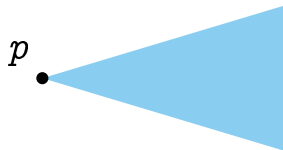
# LTS of a Process



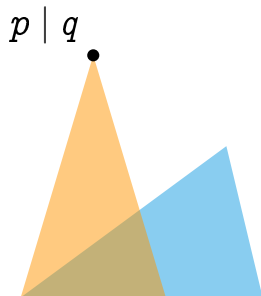
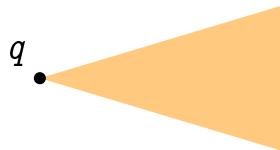
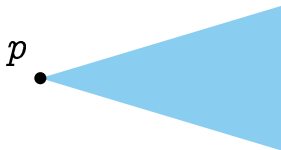
# LTS of a Process



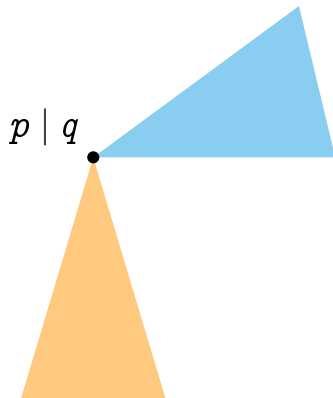
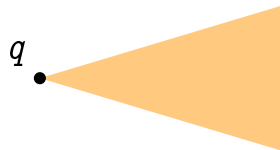
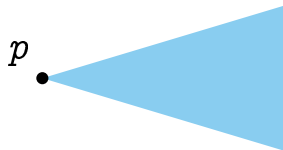
# LTS of a Process



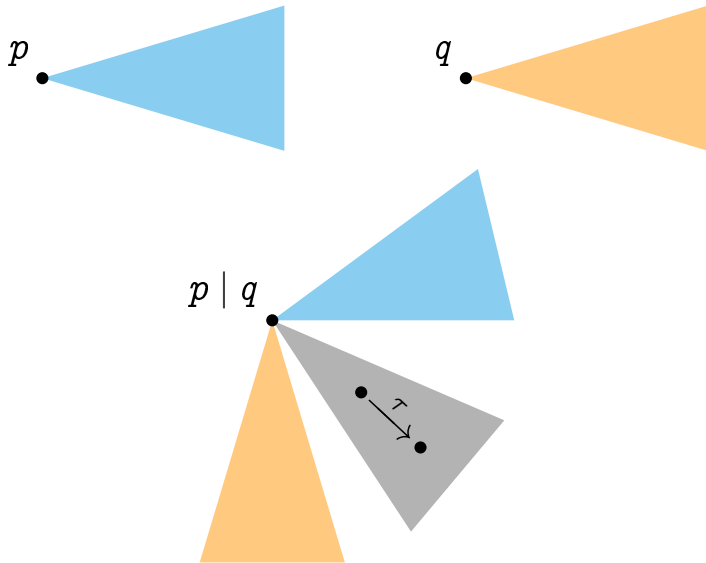
# LTS of a Process



# LTS of a Process



# LTS of a Process



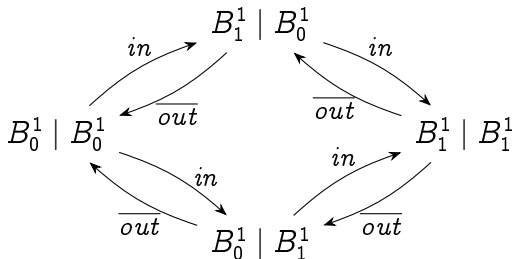
## Example: Parallel Buffer



- Recall the definitions for the buffer of capacity 1:

$$B_0^1 \triangleq in.B_1^1 \quad B_1^1 \triangleq \overline{out}.B_0^1$$

- Consider a **parallel buffer** defined as:



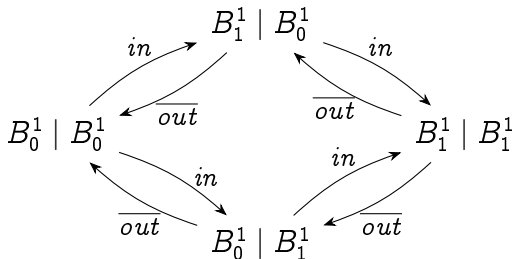
## Example: Parallel Buffer



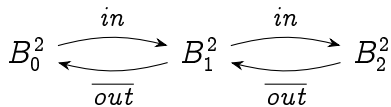
- Recall the definitions for the buffer of capacity 1:

$$B_0^1 \triangleq in.B_1^1 \quad B_1^1 \triangleq \overline{out}.B_0^1$$

- Consider a **parallel buffer** defined as:



- Compare this to the 2-capacity buffer:





$$\text{Res} \quad \frac{p \xrightarrow{\mu} q \quad \mu \notin \{\alpha, \bar{\alpha}\}}{p \backslash \alpha \xrightarrow{\mu} q \backslash \alpha}$$

- ▶ Makes  $\alpha$  private to  $p$ , i.e.,  $\alpha$  is local and not visible to the environment.
- ▶ Parallel processes in  $p$  can synchronize along  $\alpha$ .



$$\text{Res} \quad \frac{p \xrightarrow{\mu} q \quad \mu \notin \{\alpha, \bar{\alpha}\}}{p \backslash \alpha \xrightarrow{\mu} q \backslash \alpha}$$

- Makes  $\alpha$  private to  $p$ , i.e.,  $\alpha$  is local and not visible to the environment.
- Parallel processes in  $p$  can synchronize along  $\alpha$ .

## Example

Given  $P \triangleq \overline{\text{coin}}.\text{coffee}.\text{nil}$  and  $M \triangleq \text{coin} . (\overline{\text{coffee}}.\text{nil} + \overline{\text{tea}}.\text{nil})$ , we have:

$$\begin{aligned} (P \mid M) \backslash \text{coin} \backslash \text{coffee} \backslash \text{tea} &\xrightarrow{\tau} (\text{coffee}.\text{nil} \mid \overline{\text{coffee}}.\text{nil} + \overline{\text{tea}}.\text{nil}) \backslash \text{coin} \backslash \text{coffee} \backslash \text{tea} \\ &\xrightarrow{\tau} (\text{nil} \mid \text{nil}) \backslash \text{coin} \backslash \text{coffee} \backslash \text{tea} \end{aligned}$$



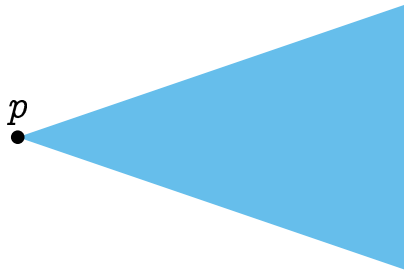
- ▶ Given  $S = \{\alpha_1, \dots, \alpha_n\}$ , we write  $p \backslash S$  instead of  $p \backslash \alpha_1 \cdots \backslash \alpha_n$ .
- ▶ We omit trailing 'nil'.

## Example

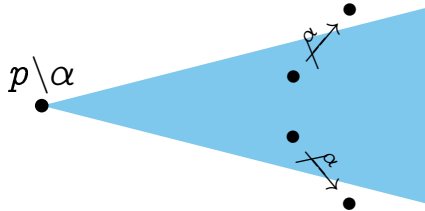
Given  $P \triangleq \overline{coin}.coffee$ ,  $M \triangleq coin.(\overline{coffee} + \overline{tea})$ ,  $S \triangleq \{coin, coffee, tea\}$ , we have:

$$(P \mid M) \backslash S \xrightarrow{\tau} (coffee \mid \overline{coffee} + \overline{tea}) \backslash S \xrightarrow{\tau} (nil \mid nil) \backslash S$$

# LTS of a Process



# LTS of a Process





$$\text{Rel} \quad \frac{p \xrightarrow{\mu} q}{p[\phi] \xrightarrow{\phi(\mu)} q[\phi]}$$

- ▶ Allows to reuse processes by renaming the action channels according to  $\phi$ .
- ▶ Conditions:  $\phi(\tau) = \tau$  and  $\phi(\bar{\lambda}) = \overline{\phi(\lambda)}$ .

$$\text{Rel} \quad \frac{p \xrightarrow{\mu} q}{p[\phi] \xrightarrow{\phi(\mu)} q[\phi]}$$

- ▶ Allows to reuse processes by renaming the action channels according to  $\phi$ .
- ▶ Conditions:  $\phi(\tau) = \tau$  and  $\phi(\bar{\lambda}) = \overline{\phi(\lambda)}$ .

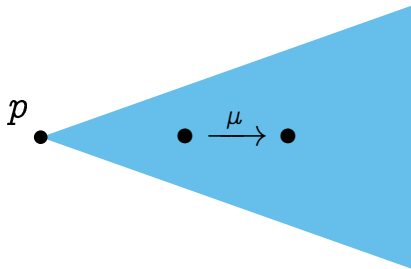
## Example

Given  $P \triangleq \overline{\text{coin}}.\text{coffee}$  and  $\phi$  defined as

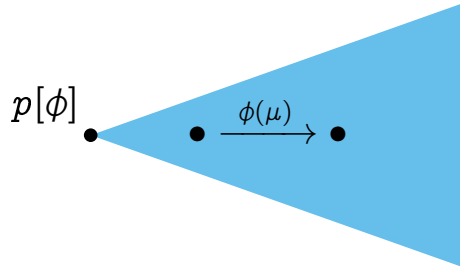
$$\phi(\text{coin}) = \text{moneta} \quad \phi(\text{coffee}) = \text{caffè}$$

We have:  $P[\phi] \xrightarrow{\overline{\text{moneta}}} \text{coffee}[\phi] \xrightarrow{\overline{\text{caffè}}} \text{nil}.$

# LTS of a Process



# LTS of a Process



# Structural Operational Semantics



$$\begin{array}{c}
 \text{act} \\
 \frac{}{\mu.p \xrightarrow{\mu} p}
 \end{array}
 \quad
 \begin{array}{c}
 \text{SumL} \\
 \frac{p_1 \xrightarrow{\mu} q}{p_1 + p_2 \xrightarrow{\mu} q}
 \end{array}
 \quad
 \begin{array}{c}
 \text{SumR} \\
 \frac{p_2 \xrightarrow{\mu} q}{p_1 + p_2 \xrightarrow{\mu} q}
 \end{array}
 \quad
 \begin{array}{c}
 \text{Rec} \\
 \frac{p[\text{rec } x. p/x] \xrightarrow{\mu} q}{\text{rec } x. p \xrightarrow{\mu} q}
 \end{array}$$
  

$$\begin{array}{c}
 \text{ParL} \\
 \frac{p_1 \xrightarrow{\mu} q_1}{p_1 \mid p_2 \xrightarrow{\mu} q_1 \mid p_2}
 \end{array}
 \quad
 \begin{array}{c}
 \text{Com} \\
 \frac{p_1 \xrightarrow{\lambda} q_1 \quad p_2 \xrightarrow{\bar{\lambda}} q_2}{p_1 \mid p_2 \xrightarrow{\tau} q_1 \mid q_2}
 \end{array}
 \quad
 \begin{array}{c}
 \text{ParR} \\
 \frac{p_2 \xrightarrow{\mu} q_2}{p_1 \mid p_2 \xrightarrow{\mu} p_1 \mid p_2}
 \end{array}$$
  

$$\begin{array}{c}
 \text{Res} \\
 \frac{p \xrightarrow{\mu} q \quad \mu \notin \{\alpha, \bar{\alpha}\}}{p \backslash \alpha \xrightarrow{\mu} q \backslash \alpha}
 \end{array}
 \quad
 \begin{array}{c}
 \text{Rel} \\
 \frac{p \xrightarrow{\mu} q}{p[\phi] \xrightarrow{\phi(\mu)} q[\phi]}
 \end{array}$$

# Linked Buffers (1)



$$\begin{array}{ll} B_0^1 \triangleq in.B_1^1 & \eta(out) = c \\ B_1^1 \triangleq \overline{out}.B_0^1 & \phi(in) = c \end{array} \quad \begin{array}{c} B_0^1[\eta] \xrightarrow{in} B_1^1[\eta] \\ \xleftarrow{\bar{c}} \end{array} \quad \begin{array}{c} B_0^1[\phi] \xrightarrow{c} B_1^1[\phi] \\ \xleftarrow{\overline{out}} \end{array}$$

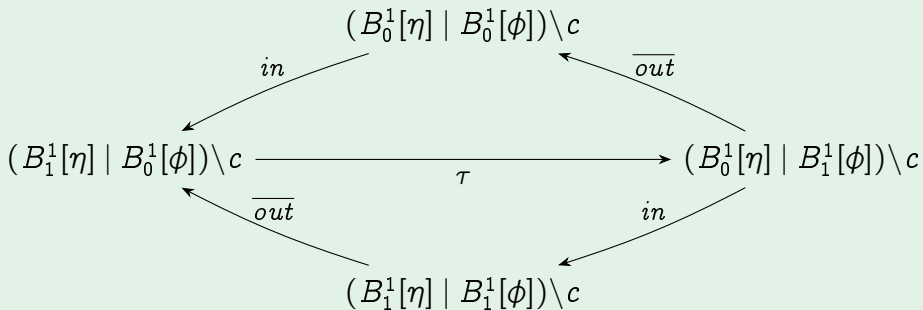
Consider the linked system  $(B_0^1[\eta] \mid B_0^1[\phi]) \setminus c$

# Linked Buffers (1)



$$\begin{array}{ll}
 B_0^1 \triangleq in.B_1^1 & \eta(out) = c \\
 B_1^1 \triangleq \overline{out}.B_0^1 & \phi(in) = c
 \end{array}
 \quad
 \begin{array}{ccc}
 B_0^1[\eta] & \xrightleftharpoons[in]{in} & B_1^1[\eta] \\
 B_0^1[\phi] & \xrightleftharpoons[out]{c} & B_1^1[\phi]
 \end{array}$$

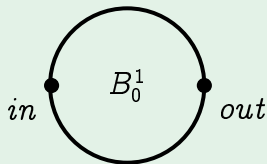
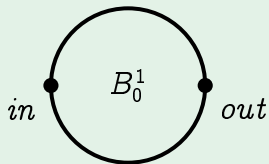
Consider the linked system  $(B_0^1[\eta] \mid B_0^1[\phi]) \setminus c$



## Linked Buffers (2)



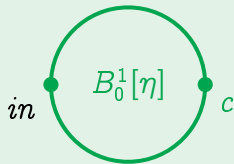
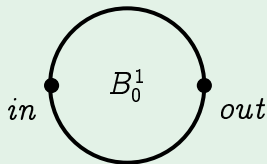
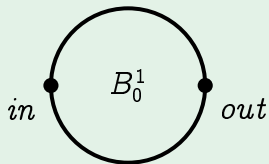
$$\begin{array}{ll} B_0^1 \triangleq in.B_1^1 & \eta(out) = c \\ B_1^1 \triangleq \overline{out}.B_0^1 & \phi(in) = c \end{array}$$
$$B_0^1[\eta] \begin{array}{c} \xrightarrow{in} \\ \xleftarrow{\bar{c}} \end{array} B_1^1[\eta] \qquad B_0^1[\phi] \begin{array}{c} \xrightarrow{c} \\ \xleftarrow{\overline{out}} \end{array} B_1^1[\phi]$$



## Linked Buffers (2)



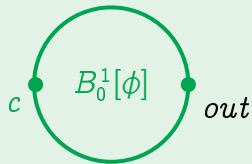
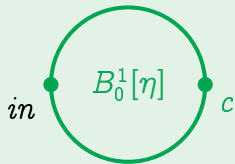
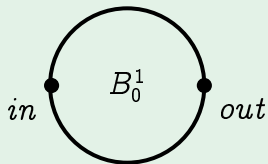
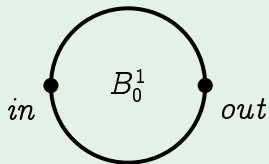
$$\begin{array}{ll} B_0^1 \triangleq in.B_1^1 & \eta(out) = c \\ B_1^1 \triangleq \overline{out}.B_0^1 & \phi(in) = c \end{array} \quad \begin{array}{c} B_0^1[\eta] \xrightarrow{in} B_1^1[\eta] \\ \xleftarrow{\bar{c}} \end{array} \quad \begin{array}{c} B_0^1[\phi] \xrightarrow{c} B_1^1[\phi] \\ \xleftarrow{\overline{out}} \end{array}$$



## Linked Buffers (2)



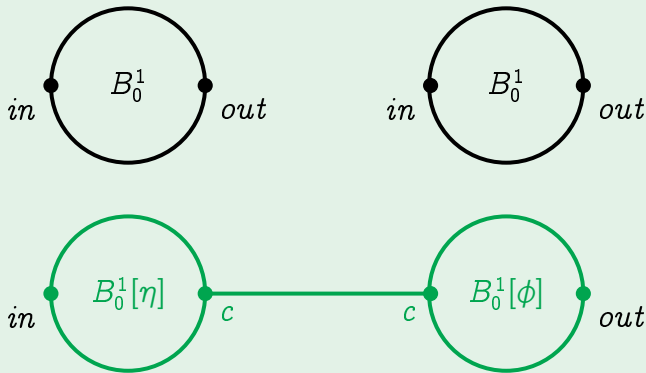
$$\begin{array}{ll} B_0^1 \triangleq in.B_1^1 & \eta(out) = c \\ B_1^1 \triangleq \overline{out}.B_0^1 & \phi(in) = c \end{array} \quad \begin{array}{c} B_0^1[\eta] \xrightarrow{in} B_1^1[\eta] \\ \xleftarrow{\bar{c}} \end{array} \quad \begin{array}{c} B_0^1[\phi] \xrightarrow{c} B_1^1[\phi] \\ \xleftarrow{\overline{out}} \end{array}$$



## Linked Buffers (2)



$$\begin{array}{ll} B_0^1 \triangleq in.B_1^1 & \eta(out) = c \\ B_1^1 \triangleq \overline{out}.B_0^1 & \phi(in) = c \end{array} \quad \begin{array}{c} B_0^1[\eta] \xrightarrow{in} B_1^1[\eta] \\ \xleftarrow{\bar{c}} \end{array} \quad \begin{array}{c} B_0^1[\phi] \xrightarrow{c} B_1^1[\phi] \\ \xleftarrow{\overline{out}} \end{array}$$



# Linked Buffers (3)



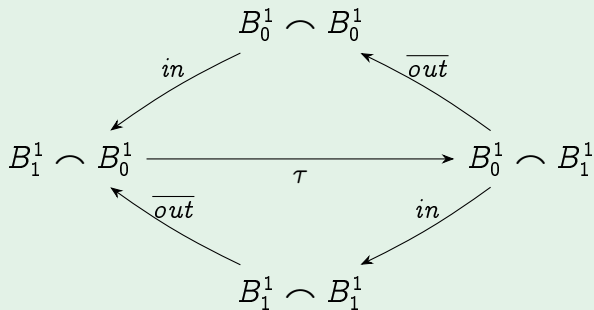
$$B_0^1 \triangleq in.B_1^1$$

$$\eta(out) = c$$

$$p \curvearrowright q \triangleq (p[\eta] \mid q[\phi]) \setminus c$$

$$B_1^1 \triangleq \overline{out}.B_0^1$$

$$\phi(in) = c$$



# Linked Boolean Buffers



$$B_{\emptyset} \triangleq in_{tt}.B_{tt} + in_{ff}.B_{ff}$$

$$B_{tt} \triangleq \overline{out}_{tt}.B_{\emptyset}$$

$$B_{ff} \triangleq \overline{out}_{ff}.B_{\emptyset}$$

$$\eta(out_{ff}) = c_{ff}$$

$$\phi(in_{ff}) = c_{ff}$$

$$\eta(out_{tt}) = c_{tt}$$

$$\phi(in_{tt}) = c_{tt}$$

# Linked Boolean Buffers



$$B_{\emptyset} \triangleq in_{tt}.B_{tt} + in_{ff}.B_{ff}$$

$$B_{tt} \triangleq \overline{out_{tt}}.B_{\emptyset}$$

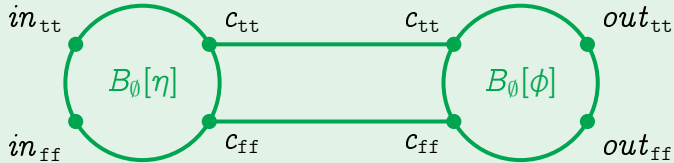
$$B_{ff} \triangleq \overline{out_{ff}}.B_{\emptyset}$$

$$\eta(out_{ff}) = c_{ff}$$

$$\eta(out_{tt}) = c_{tt}$$

$$\phi(in_{ff}) = c_{ff}$$

$$\phi(in_{tt}) = c_{tt}$$



# Linked Boolean Buffers: Exercise



$$B_{\emptyset} \triangleq in_{tt}.B_{tt} + in_{ff}.B_{ff}$$

$$\eta(out_{ff}) = c_{ff}$$

$$\eta(out_{tt}) = c_{tt}$$

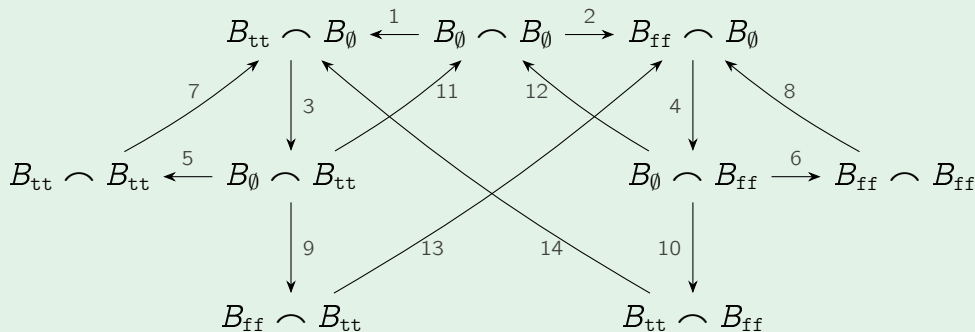
$$B_{tt} \triangleq \overline{out}_{tt}.B_{\emptyset}$$

$$\phi(in_{ff}) = c_{ff}$$

$$\phi(in_{tt}) = c_{tt}$$

$$B_{ff} \triangleq \overline{out}_{ff}.B_{\emptyset}$$

$$p \curvearrowright q \triangleq (p[\eta] \mid q[\phi]) \setminus \{c_{ff}, c_{tt}\}$$



- ▶ A web-based tool for specifying and verifying processes expressed in CCS
- ▶ In particular, you can explore the LTS of the given processes
- ▶ Good didactic approach to understand the semantics of processes
- ▶ URL: <https://caal.cs.aau.dk/>

# CCS with Value Passing



- We have seen actions  $\alpha$  and  $\bar{\alpha}$ . A convenient extension, in which  $\alpha$  acts as a channel for sending and receiving a value:

$\text{send}_v$

$\text{recv}_v$

$$\frac{}{\alpha!v.p \xrightarrow{\bar{\alpha}_v} p}$$

$$\frac{}{\alpha?x.p \xrightarrow{\alpha_v} p[v/x]}$$

# CCS with Value Passing



- We have seen actions  $\alpha$  and  $\bar{\alpha}$ . A convenient extension, in which  $\alpha$  acts as a channel for sending and receiving a value:

$$\begin{array}{c} \text{send}_v \\ \hline \alpha!v.p \xrightarrow{\bar{\alpha}_v} p \end{array} \qquad \begin{array}{c} \text{recv}_v \\ \hline \alpha?x.p \xrightarrow{\alpha_v} p[v/x] \end{array}$$

- Assuming a finite set of values  $V \triangleq \{v_1, \dots, v_n\}$ , the value-passing constructs are **representable** in plain CCS.
- Formally, we define a **translation**. Main cases:

$$\llbracket \alpha!v_i.p \rrbracket = \bar{\alpha}_{v_i}.\llbracket p \rrbracket$$

$$\llbracket \alpha?x.p \rrbracket = \alpha_{v_1}.\llbracket p[v_1/x] \rrbracket + \dots + \alpha_{v_n}.\llbracket p[v_n/x] \rrbracket$$

# CCS with Value Passing



- ▶ We have seen actions  $\alpha$  and  $\bar{\alpha}$ . A convenient extension, in which  $\alpha$  acts as a channel for sending and receiving a value:

$$\begin{array}{c} \text{send}_v \qquad \qquad \text{recv}_v \\ \hline \alpha!v.p \xrightarrow{\bar{\alpha}_v} p \qquad \alpha?x.p \xrightarrow{\alpha_v} p[v/x] \end{array}$$

- ▶ Assuming a finite set of values  $V \triangleq \{v_1, \dots, v_n\}$ , the value-passing constructs are **representable** in plain CCS.
- ▶ Formally, we define a **translation**. Main cases:

$$\begin{aligned} \llbracket \alpha!v_i.p \rrbracket &= \bar{\alpha}_{v_i}.\llbracket p \rrbracket \\ \llbracket \alpha?x.p \rrbracket &= \alpha_{v_1}.\llbracket p[v_1/x] \rrbracket + \dots + \alpha_{v_n}.\llbracket p[v_n/x] \rrbracket \end{aligned}$$

- ▶ That is, with finite  $V$ , value passing is a syntactic sugar.

## Exercise: Find the LTS (1)



$$P \triangleq \text{coin}.\overline{\text{coffee}} + \text{coin}.\overline{\text{tea}}$$

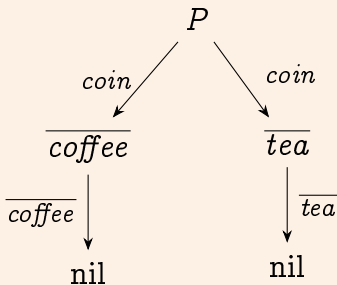
$$Q \triangleq \text{coin}.\overline{(\text{coffee} + \text{tea})}$$

## Exercise: Find the LTS (1)



$$P \triangleq \text{coin}.\overline{\text{coffee}} + \text{coin}.\overline{\text{tea}}$$

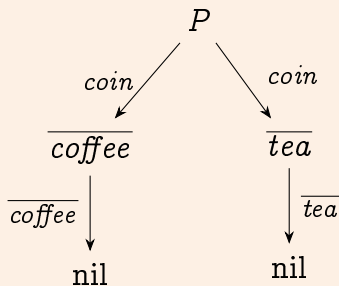
$$Q \triangleq \text{coin}.\overline{(\text{coffee} + \text{tea})}$$



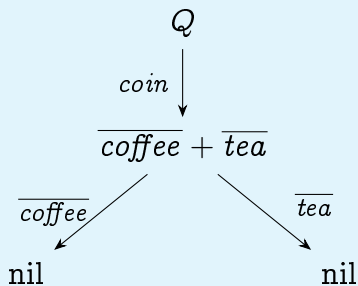
## Exercise: Find the LTS (1)



$$P \triangleq \text{coin}.\overline{\text{coffee}} + \text{coin}.\overline{\text{tea}}$$



$$Q \triangleq \text{coin}.\overline{(\text{coffee} + \text{tea})}$$



## Exercise: Find the LTS (2)



$$P \triangleq \text{rec } x. \alpha.x + \text{rec } x. \beta.x$$

$$Q \triangleq \text{rec } x. (\alpha.x + \beta.x)$$

$$R \triangleq \text{rec } x. (\alpha.x + \beta.\text{nil})$$

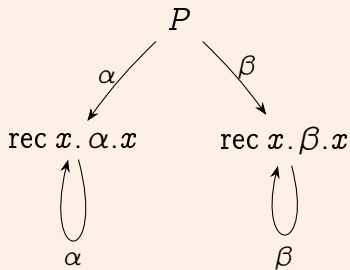
## Exercise: Find the LTS (2)



$$P \triangleq \text{rec } x. \alpha.x + \text{rec } x. \beta.x$$

$$Q \triangleq \text{rec } x. (\alpha.x + \beta.x)$$

$$R \triangleq \text{rec } x. (\alpha.x + \beta.\text{nil})$$



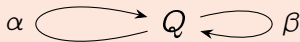
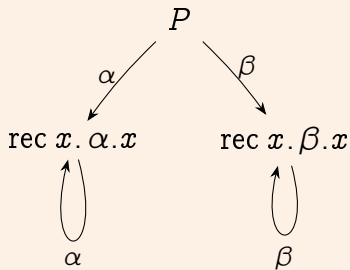
## Exercise: Find the LTS (2)



$$P \triangleq \text{rec } x. \alpha.x + \text{rec } x. \beta.x$$

$$Q \triangleq \text{rec } x. (\alpha.x + \beta.x)$$

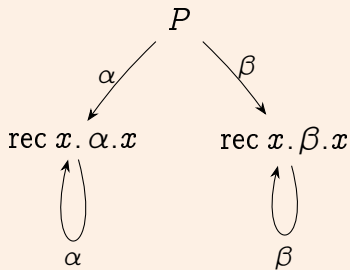
$$R \triangleq \text{rec } x. (\alpha.x + \beta.\text{nil})$$



## Exercise: Find the LTS (2)



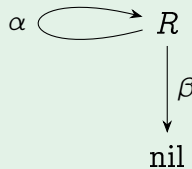
$$P \triangleq \text{rec } x. \alpha.x + \text{rec } x. \beta.x$$



$$Q \triangleq \text{rec } x. (\alpha.x + \beta.x)$$



$$R \triangleq \text{rec } x. (\alpha.x + \beta.\text{nil})$$



## Exercise: Find the LTS (3)



$$T \triangleq \text{rec } x. \alpha. (\text{nil} \mid x) + \beta. \text{nil}$$

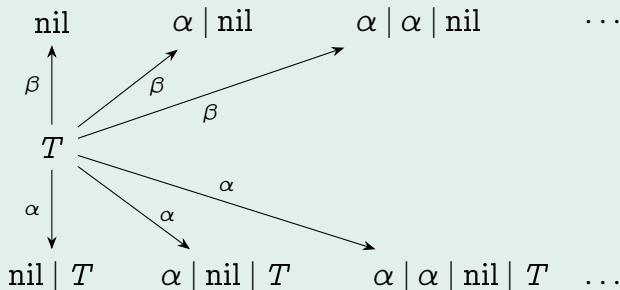
$$T \triangleq (\alpha \mid T) + \beta$$

## Exercise: Find the LTS (3)



$$T \triangleq \text{rec } x. \alpha.(\text{nil} \mid x) + \beta.\text{nil}$$

$$T \triangleq (\alpha \mid T) + \beta$$

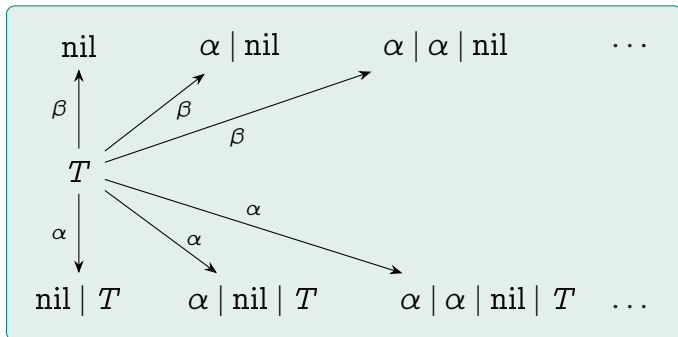


## Exercise: Find the LTS (3)



$$T \triangleq \text{rec } x. \alpha.(\text{nil} \mid x) + \beta.\text{nil}$$

$$T \triangleq (\alpha \mid T) + \beta$$



Suggestion: Try running this process in CAAL!

## Exercise: Find the LTS (4)



$$U \triangleq \text{rec } x. \alpha \mid \beta.x$$

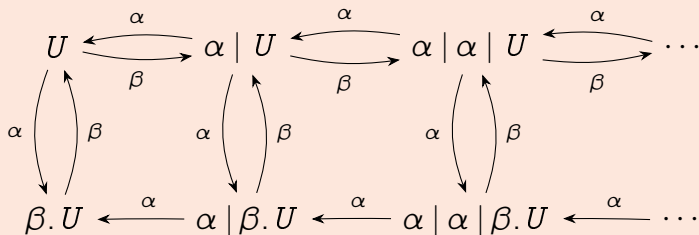
$$U \triangleq \alpha \mid (\beta.U)$$

# Exercise: Find the LTS (4)



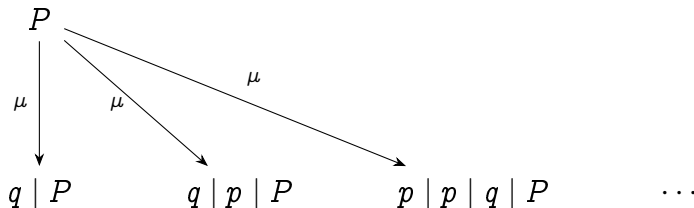
$$U \triangleq \text{rec } x. \alpha \mid \beta.x$$

$$U \triangleq \alpha \mid (\beta.U)$$





- ▶ As we have seen, the allowed form of recursion is very general. There are processes with infinitely many outgoing transitions
- ▶ Suppose  $p \xrightarrow{\mu} q$  and take  $P \triangleq \text{rec } x. p \mid x$  (that is  $P \triangleq p \mid P$ ). We have:



- ▶ We would like to avoid this kind of **infinitely branching** processes.

# Guarded Processes



Processes in which process variables occur under a prefix. That is, recursion occurs **guarded** by some action.

# Guarded Processes



Processes in which process variables occur under a prefix. That is, recursion occurs **guarded** by some action.

Guarded?

► `rec  $x$ .  $x$`

# Guarded Processes



Processes in which process variables occur under a prefix. That is, recursion occurs **guarded** by some action.

Guarded?

► `rec  $x$ .  $x$`

**X**

# Guarded Processes



Processes in which process variables occur under a prefix. That is, recursion occurs **guarded** by some action.

Guarded?

►  $\text{rec } x. x$

**X**

►  $\text{rec } x. \alpha. \text{rec } y. x$

# Guarded Processes



Processes in which process variables occur under a prefix. That is, recursion occurs **guarded** by some action.

Guarded?

- ▶  $\text{rec } x. x$  **X**
- ▶  $\text{rec } x. \alpha. \text{rec } y. x$  **✓**



Processes in which process variables occur under a prefix. That is, recursion occurs **guarded** by some action.

Guarded?

- ▶  $\text{rec } x. x$  ✗
- ▶  $\text{rec } x. \alpha. \text{rec } y. x$  ✓
- ▶  $\text{rec } x. \alpha. \text{rec } y. x + y$



Processes in which process variables occur under a prefix. That is, recursion occurs **guarded** by some action.

Guarded?

- |                                                 |          |
|-------------------------------------------------|----------|
| ▶ $\text{rec } x. x$                            | $\times$ |
| ▶ $\text{rec } x. \alpha. \text{rec } y. x$     | ✓        |
| ▶ $\text{rec } x. \alpha. \text{rec } y. x + y$ | $\times$ |



Processes in which process variables occur under a prefix. That is, recursion occurs **guarded** by some action.

Guarded?

- ▶  $\text{rec } x. x$   $\times$
- ▶  $\text{rec } x. \alpha. \text{rec } y. x$  ✓
- ▶  $\text{rec } x. \alpha. \text{rec } y. x + y$   $\times$
- ▶  $\text{rec } x. \alpha. \text{rec } y. x \mid y$



Processes in which process variables occur under a prefix. That is, recursion occurs **guarded** by some action.

Guarded?

- |                                                    |          |
|----------------------------------------------------|----------|
| ▶ $\text{rec } x. x$                               | $\times$ |
| ▶ $\text{rec } x. \alpha. \text{rec } y. x$        | ✓        |
| ▶ $\text{rec } x. \alpha. \text{rec } y. x + y$    | $\times$ |
| ▶ $\text{rec } x. \alpha. \text{rec } y. x \mid y$ | $\times$ |



Processes in which process variables occur under a prefix. That is, recursion occurs **guarded** by some action.

Guarded?

- |                                                           |          |
|-----------------------------------------------------------|----------|
| ► $\text{rec } x. x$                                      | $\times$ |
| ► $\text{rec } x. \alpha. \text{rec } y. x$               | ✓        |
| ► $\text{rec } x. \alpha. \text{rec } y. x + y$           | $\times$ |
| ► $\text{rec } x. \alpha. \text{rec } y. x \mid y$        | $\times$ |
| ► $\text{rec } x. \alpha. \text{rec } y. x \mid \beta. y$ | ✓        |



- ▶ Syntax of CCS
- ▶ Operational Semantics (LTS)
- ▶ Linked Buffers
- ▶ Infinitely Branching and Guarded Processes

**Next:** Behavioral equivalences: *When  $P$  and  $Q$  can be considered 'equivalent'?*



The End